

Exercise 1: Control Structures

Scenario 1: The bank wants to apply a discount to loan interest rates for customers above 60 years old.

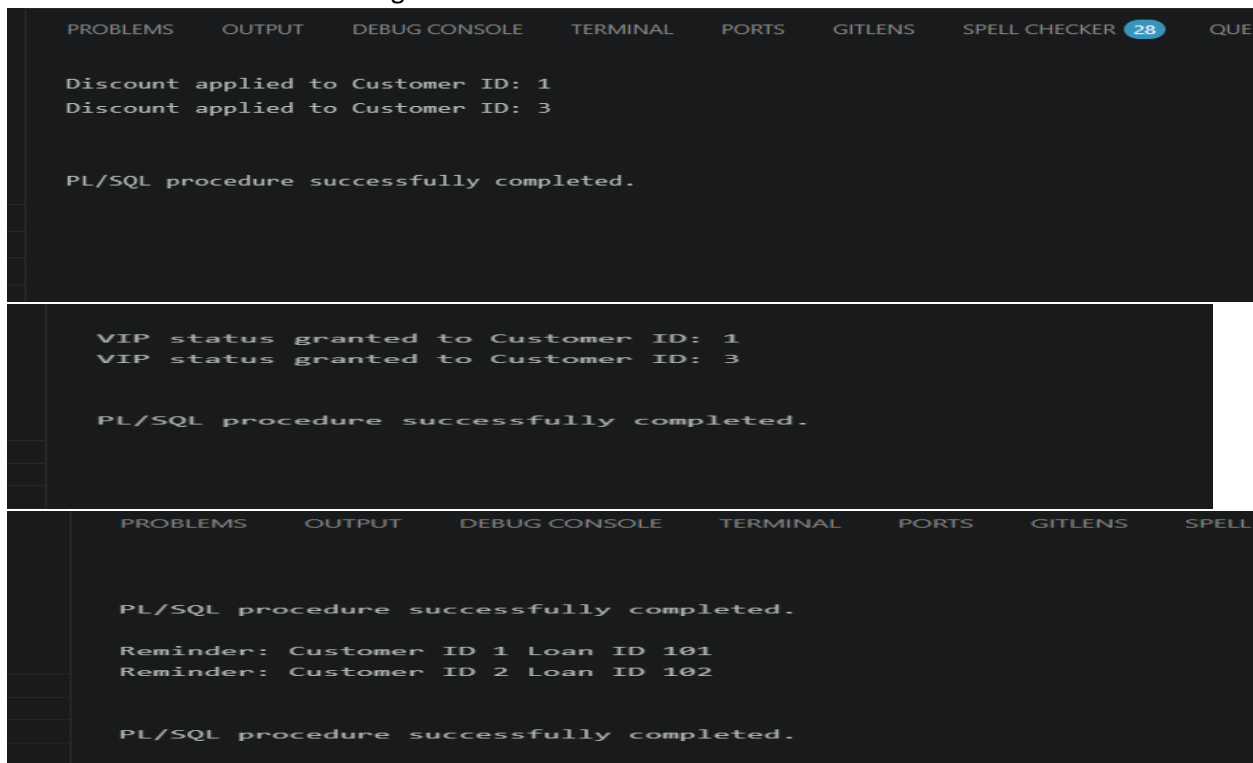
- **Question:** Write a PL/SQL block that loops through all customers, checks their age, and if they are above 60, apply a 1% discount to their current loan interest rates.

Scenario 2: A customer can be promoted to VIP status based on their balance.

- **Question:** Write a PL/SQL block that iterates through all customers and sets a flag `IsVIP` to `TRUE` for those with a balance over \$10,000.

Scenario 3: The bank wants to send reminders to customers whose loans are due within the next 30 days.

- **Question:** Write a PL/SQL block that fetches all loans due in the next 30 days and prints a reminder message for each customer.



The image displays three screenshots of a SQL IDE terminal window, showing the execution of PL/SQL procedures for the scenarios in Exercise 1. The terminal has a dark background with light-colored text. The top screenshot shows the output for Scenario 1, where a discount is applied to customers aged 60 and above. The middle screenshot shows the output for Scenario 2, where VIP status is granted to customers with a balance over \$10,000. The bottom screenshot shows the output for Scenario 3, where reminders are sent to customers whose loans are due within the next 30 days.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 28 QUE

Discount applied to Customer ID: 1
Discount applied to Customer ID: 3

PL/SQL procedure successfully completed.

VIP status granted to Customer ID: 1
VIP status granted to Customer ID: 3

PL/SQL procedure successfully completed.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL

PL/SQL procedure successfully completed.
Reminder: Customer ID 1 Loan ID 101
Reminder: Customer ID 2 Loan ID 102

PL/SQL procedure successfully completed.
```

Exercise 2: Error Handling

Scenario 1: Handle exceptions during fund transfers between accounts.

- **Question:** Write a stored procedure **SafeTransferFunds** that transfers funds between two accounts. Ensure that if any error occurs (e.g., insufficient funds), an appropriate error message is logged and the transaction is rolled back.

Scenario 2: Manage errors when updating employee salaries.

- **Question:** Write a stored procedure **UpdateSalary** that increases the salary of an employee by a given percentage. If the employee ID does not exist, handle the exception and log an error message.

Scenario 3: Ensure data integrity when adding a new customer.

- **Question:** Write a stored procedure **AddNewCustomer** that inserts a new customer into the Customers table. If a customer with the same ID already exists, handle the exception by logging an error and preventing the insertion.

```
Commit complete.

Procedure SAFETRANSFERFUNDS compiled
Funds Transferred Successfully

PL/SQL procedure successfully completed.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  SPELL C

Commit complete.

Procedure UPDATESALARY compiled
Salary Updated Successfully

PL/SQL procedure successfully completed.
```

```
Procedure ADDNEWCUSTOMER compiled
Customer Added Successfully

PL/SQL procedure successfully completed.
```

Exercise 3: Stored Procedures

Scenario 1: The bank needs to process monthly interest for all savings accounts.

- **Question:** Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all savings accounts by applying an interest rate of 1% to the current balance.

Scenario 2: The bank wants to implement a bonus scheme for employees based on their performance.

- **Question:** Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

Scenario 3: Customers should be able to transfer funds between their accounts.

- **Question:** Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  SPELL

Commit complete.

Procedure PROCESMONTHLYINTEREST compiled

Monthly Interest Applied Successfully

PL/SQL procedure successfully completed.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

Commit complete.

Procedure UPDATEEMPLOYEEBONUS compiled

Bonus Updated Successfully

PL/SQL procedure successfully completed.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

Procedure TRANSFERFUNDS compiled

Funds Transferred Successfully

PL/SQL procedure successfully completed.
```

Exercise 4: Functions

Scenario 1: Calculate the age of customers for eligibility checks.

- **Question:** Write a function **CalculateAge** that takes a customer's date of birth as input and returns their age in years.

Scenario 2: The bank needs to compute the monthly installment for a loan.

- **Question:** Write a function **CalculateMonthlyInstallment** that takes the loan amount, interest rate, and loan duration in years as input and returns the monthly installment amount.

Scenario 3: Check if a customer has sufficient balance before making a transaction.

- **Question:** Write a function **HasSufficientBalance** that takes an account ID and an amount as input and returns a boolean indicating whether the account has at least the specified amount.

The screenshot displays the SQL Developer interface with three panels. The top panel shows a query result for the 'AGE' column, with one row containing the value 23. The middle panel shows a query result for the 'MONTHLY_EMI' column, with one row containing the value 10138.2. The bottom panel shows the execution output of the 'HasSufficientBalance' function, which returns 'Sufficient Balance'.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL C

All rows fetched: 1 in 0.078 seconds

	AGE
1	23

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER

All rows fetched: 1 in 0.048 seconds

	MONTHLY_EMI
1	10138.2

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL C

Commit complete.

Function HASSUFFICIENTBALANCE compiled

Sufficient Balance

PL/SQL procedure successfully completed.

Exercise 5: Triggers

Scenario 1: Automatically update the last modified date when a customer's record is updated.

- **Question:** Write a trigger **UpdateCustomerLastModified** that updates the LastModified column of the Customers table to the current date whenever a customer's record is updated.

Scenario 2: Maintain an audit log for all transactions.

- **Question:** Write a trigger **LogTransaction** that inserts a record into an AuditLog table whenever a transaction is inserted into the Transactions table.

Scenario 3: Enforce business rules on deposits and withdrawals.

- **Question:** Write a trigger **CheckTransactionRules** that ensures withdrawals do not exceed the balance and deposits are positive before inserting a record into the Transactions table.

The image displays three screenshots of a database query tool interface, likely SQL Developer, showing query results for different tables.

Screenshot 1 (Top): Shows the 'CUSTOMER' table with 4 rows fetched in 0.018 seconds. The columns are CUSTOMER_ID, AGE, INTEREST_RATE, BALANCE, and ISVIP.

	CUSTOMER_ID	AGE	INTEREST_RATE	BALANCE	ISVIP
1	1	65	9	15000	TRUE
2	2	45	12	8000	FALSE
3	3	70	10	20000	TRUE
4	10	30	12	15000	FALSE

Screenshot 2 (Middle): Shows the 'TRANSACTION_LOG' table with 1 row fetched in 0.024 seconds. The columns are LOG_ID, TRANSACTION_ID, and ACTION_DATE.

	LOG_ID	TRANSACTION_ID	ACTION_DATE
1	1	1	25-06-26

Screenshot 3 (Bottom): Shows the 'TRANSACTION' table with 1 row fetched in 0.018 seconds. The columns are TRANSACTION_ID, ACCOUNT_ID, TRANSACTION_TYPE, and AMOUNT.

	TRANSACTION_ID	ACCOUNT_ID	TRANSACTION_TYPE	AMOUNT
1	2	101	DEPOSIT	5000

Exercise 6: Cursors

Scenario 1: Generate monthly statements for all customers.

- **Question:** Write a PL/SQL block using an explicit cursor **GenerateMonthlyStatements** that retrieves all transactions for the current month and prints a statement for each customer.

Scenario 2: Apply annual fee to all accounts.

- **Question:** Write a PL/SQL block using an explicit cursor **ApplyAnnualFee** that deducts an annual maintenance fee from the balance of all accounts.

Scenario 3: Update the interest rate for all loans based on a new policy.

- **Question:** Write a PL/SQL block using an explicit cursor **UpdateLoanInterestRates** that fetches all loans and updates their interest rates based on the new policy.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPELL CHECKER 4 QUERY RESULT SCRIPT OUTPUT SQL HIS						
All rows fetched: 2 in 0.022 seconds						
	TRANSACTION_ID	ACCOUNT_ID	TRANSACTION_TYPE	AMOUNT	TRANSACTION_DATE	
1	1	101	DEPOSIT	5000	25-06-26	
2	2	101	DEPOSIT	5000	25-06-26	

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS SPEL						
All rows fetched: 3 in 0.036 seconds						
	ACCOUNT_ID	BALANCE				
1	1	5500				
2	2	8500				
3	101	9500				

All rows fetched: 3 in 0.026 seconds						
	LOAN_ID	CUSTOMER_ID	DUE_DATE	AMOUNT	INTEREST_RATE	
1	101	1	04-07-26	5000	6.5	
2	102	2	19-07-26	7000	6.5	
3	103	3	03-08-26	9000	6.5	

Exercise 7: Packages

Scenario 1: Group all customer-related procedures and functions into a package.

- **Question:** Create a package **CustomerManagement** with procedures for adding a new customer, updating customer details, and a function to get customer balance.

Scenario 2: Create a package to manage employee data.

- **Question:** Write a package **EmployeeManagement** with procedures to hire new employees, update employee details, and a function to calculate annual salary.

Scenario 3: Group all account-related operations into a package.

- **Question:** Create a package **AccountOperations** with procedures for opening a new account, closing an account, and a function to get the total balance of a customer across all accounts.

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	GITLENS	SPELL CHECKER	5	QUERY RESULTS
All rows fetched: 4 in 0.027 seconds								
	CUSTOMER_ID	AGE	INTEREST_RATE	BALANCE	ISVIP			
1	1	65	9	15000	TRUE			
2	2	45	12	8000	FALSE			
3	3	70	10	20000	TRUE			
4	10	30	12	15000	FALSE			

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	GITLENS	SPELL
All rows fetched: 2 in 0.030 seconds						
	EMP_ID	EMP_NAME	SALARY	DEPARTMENT		
1	101	Ravi	36300	IT		
2	102	Sita	44000	IT		

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	GITLENS	SPELL CHECK
All rows fetched: 1 in 0.030 seconds						
	ACCOUNTOPERATIONS.GETTOTALBALANCE(999)					
1	25000					

Schema to be Created

```
CREATE TABLE Customers (
  CustomerID NUMBER PRIMARY KEY,
  Name VARCHAR2(100),
  DOB DATE,
  Balance NUMBER,
  LastModified DATE
);
```

```
CREATE TABLE Accounts (
  AccountID NUMBER PRIMARY KEY,
  CustomerID NUMBER,
  AccountType VARCHAR2(20),
  Balance NUMBER,
  LastModified DATE,
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
```

);

```
CREATE TABLE Transactions (  
    TransactionID NUMBER PRIMARY KEY,  
    AccountID NUMBER,  
    TransactionDate DATE,  
    Amount NUMBER,  
    TransactionType VARCHAR2(10),  
    FOREIGN KEY (AccountID) REFERENCES Accounts(AccountID)  
);
```

```
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    LoanAmount NUMBER,  
    InterestRate NUMBER,  
    StartDate DATE,  
    EndDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE Employees (  
    EmployeeID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100),  
    Position VARCHAR2(50),  
    Salary NUMBER,  
    Department VARCHAR2(50),  
    HireDate DATE  
);
```

Example Scripts for Sample Data Insertion

```
INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)  
VALUES (1, 'John Doe', TO_DATE('1985-05-15', 'YYYY-MM-DD'), 1000, SYSDATE);
```

```
INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)  
VALUES (2, 'Jane Smith', TO_DATE('1990-07-20', 'YYYY-MM-DD'), 1500, SYSDATE);
```

```
INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)  
VALUES (1, 1, 'Savings', 1000, SYSDATE);
```

```
INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance, LastModified)  
VALUES (2, 2, 'Checking', 1500, SYSDATE);
```



```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
VALUES (1, 1, SYSDATE, 200, 'Deposit');
```

```
INSERT INTO Transactions (TransactionID, AccountID, TransactionDate, Amount, TransactionType)
VALUES (2, 2, SYSDATE, 300, 'Withdrawal');
```

```
INSERT INTO Loans (LoanID, CustomerID, LoanAmount, InterestRate, StartDate, EndDate)
VALUES (1, 1, 5000, 5, SYSDATE, ADD_MONTHS(SYSDATE, 60));
```

```
INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (1, 'Alice Johnson', 'Manager', 70000, 'HR', TO_DATE('2015-06-15', 'YYYY-MM-DD'));
```

```
INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department, HireDate)
VALUES (2, 'Bob Brown', 'Developer', 60000, 'IT', TO_DATE('2017-03-20', 'YYYY-MM-DD'));
```